Website Builder Backend Workflow Architecture

End-to-end architecture, backend support, runtime agents, dynamic agents, tools, validation, preview, QA, persistence, and projection layers.

This document reflects the current backend-led website builder implementation. Gemini controls routing, planning, and artifact proposal. Python remains the execution authority for identity binding, tool validation, preview builds, QA, file writes, local sync, rollback, telemetry, and persistence.

End-to-End Runtime Flow

User -> React workspace -> FastAPI /api/projects/{id}/generate or /generate-stream

run_generation_pipeline -> telemetry -> project load -> Gemini control/artifact providers -> agent_run

WorktualGenerationOrchestrator -> route intent -> optional confirmation -> stage graph

execute_real_agent_runtime_loop -> supervised legal actions -> backend tool registry

read files -> load memory -> analyze/plan -> dynamic specialists -> code/scoped update

validate artifact -> staged Vite preview -> visual QA -> WRITE_PROJECT_FILES

local folder sync -> memory persistence -> A2A/ADK/LangChain projections -> response

Primary Source Map

Area	Primary files
API entry points	backend/main.py
Generation pipeline	backend/api/generation.py, backend/api/generation_stream.py
Orchestrator	backend/llm/orchestration/runner.py, backend/llm/orchestrator.py
Real runtime loop	backend/llm/agent_runtime_loop.py, backend/llm/agent_runtime/*
Runtime actions	backend/llm/agent_runtime/actions/*
Supervisor policy	backend/llm/agent_runtime/supervision/__init__.py
Backend tools	backend/agentic/tools/registry.py, handlers.py, validators.py
Dynamic agents	backend/llm/dynamic_agenting/*, backend/llm/dynamic_agents.py
A2A communication	backend/llm/a2a/*, backend/llm/a2a_communication.py
ADK projection	backend/llm/google_adk_runtime/*
LangChain projection	backend/llm/langchain_runtime_impl/*

Area	Primary files
Validation	backend/llm/artifacts/*
Preview/runtime	backend/runtime.py, backend/visual_qa/*
Storage/local workspace	backend/storage/*, backend/local_workspace/*, backend/api/local_workspaces.py
Audit/runtime persistence	backend/audit_logging/*, backend/agentic/runtime_persistence/*

End-to-End Flowchart

Project + Prompt Entry

1. User creates/selects project.
2. Backend project or linked local folder is resolved.
3. Existing local files are pulled and validated, or an empty workspace is allowed.
4. User submits prompt.
5. FastAPI validates prompt and calls run_generation_pipeline.

Pipeline + Orchestrator

6. Telemetry context and persistent agent_run are created.
7. GeminiProvider is used for both control and artifact roles.
8. WorktualGenerationOrchestrator routes intent.
9. Pending confirmation is evaluated; high-impact generation can pause before writes.
10. Conversation-only turns return assistant text and never write files.

Agent Runtime Loop

11. Supervisor chooses only legal next actions from current state.
12. READ_PROJECT_FILES and LOAD_PROJECT_MEMORY run first.
13. Update requests run update analysis and may choose scoped update mode.
14. New generation runs prompt analysis, dynamic planning, planner, specialists, UX, accessibility, and code generation.
15. Scoped updates use SEARCH/REPLACE style edits against Python-approved files.

Backend Authority + Completion

16. Dynamic patches are integrated only after Python validation.
17. VALIDATE_PROJECT_ARTIFACT checks contract, files, paths, theme, and React safety.
18. BUILD_STAGED_PROJECT_PREVIEW builds candidate files before commit.
19. RUN_PREVIEW_VISUAL_QA checks build logs and browser/runtime integrity.
20. WRITE_PROJECT_FILES commits; linked local folders are pushed; memory and runtime records are persisted.

Runtime Agents Present in Backend

Agent	Mode	Backend responsibility
Intent Router Agent	diagnostic	Classifies the user turn and selects conversation, clarification, generation, or update branch.
Conversation Agent	descriptive	Replies to greeting or missing-detail turns without creating files.
Supervisor Agent	diagnostic	Selects legal runtime actions from state and rejects DONE until completion proof is satisfied.
Prompt Analyst Agent	descriptive	Extracts website goal, audience, brand cues, sections, and context.
Update Analysis Agent	diagnostic	Classifies existing-project updates into deterministic patch, targeted patch, feature patch, or full workflow.
Planner Agent	predictive	Plans section order, interactions, content hierarchy, conversion path, and implementation priorities.
Agent Registry Agent	diagnostic	Creates/reuses dynamic agent workflow plans and executes specialist assignments.
UX Review Agent	diagnostic	Reviews workflow, conversion clarity, responsive layout, content density, and design risks.
Accessibility Agent	diagnostic	Reviews semantic structure, contrast, keyboard flow, labels, ARIA, and mobile text fit.
Code Agent	prescriptive	Generates strict project artifact JSON and commits only after backend validation and QA.
Scoped Update Agent	prescriptive	Applies scoped patches to existing files selected by update analysis.
Validation Agent	diagnostic	Validates generated website structure, paths, required files, theme, and file safety.
Preview Agent	diagnostic	Builds candidate files in staged Vite preview before final write.
Visual QA Agent	diagnostic	Runs backend preview integrity and browser/runtime QA checks.
Repair Agent	prescriptive	Repairs validation, artifact, scoped patch, or preview failures within repair budget.
Memory Agent	descriptive	Loads and persists project/user memory and dynamic agent lifecycle snapshots.

Runtime Action Map

Action	Owning agent	Purpose
READ_PROJECT_FILES	Memory Agent	Read current project files from backend store or pull from linked local folder.
LOAD_PROJECT_MEMORY	Memory Agent	Load project/user memory and dynamic agent context.
RUN_UPDATE_ANALYST	Update Analysis Agent	Choose update mode and candidate files for existing-project work.
RUN_SCOPED_UPDATE_AGENT	Scoped Update Agent	Patch only approved existing files for targeted or feature updates.
RUN_PROMPT_ANALYST	Prompt Analyst Agent	Build structured website brief for generation.
RUN_DYNAMIC_AGENT_PLANNER	Agent Registry Agent	Build guarded dynamic workflow and assign tasks to agents.
RUN_DYNAMIC_SPECIALISTS	Agent Registry Agent	Run domain/capability specialists and collect bounded candidate changes.
RUN_PLANNER	Planner Agent	Create implementation and layout plan.

Action	Owning agent	Purpose
RUN_UX_REVIEW_AGENT	UX Review Agent	Review plan for usability and responsive quality.
RUN_ACCESSIBILITY_AGENT	Accessibility Agent	Review plan for accessibility and mobile fit.
RUN_CODE_AGENT	Code Agent	Generate complete artifact/files.
RUN_DYNAMIC_PATCH_INTEGRATOR	Code Generator Agent	Integrate accepted dynamic-agent candidate changes.
RUN_REPAIR_AGENT	Repair Agent	Repair failed artifact, validation, build, or QA results.
VALIDATE_PROJECT_ARTIFACT	Validation Agent	Validate strict website artifact before preview/write.
BUILD_STAGED_PROJECT_PREVIEW	Preview Agent	Build candidate files before final commit.
RUN_PREVIEW_VISUAL_QA	Visual QA Agent	Check staged preview readiness and runtime/browser issues.
WRITE_PROJECT_FILES	Code Agent	Replace project files after validation, preview, and QA pass.
PERSIST_PROJECT_MEMORY	Memory Agent	Persist final project memory and runtime checkpoint.
DONE	Supervisor Agent	Stop only after completion proof is satisfied.

Dynamic Agents Flow

Dynamic Agent Lifecycle

Prompt/update analysis -> capability tasks -> registry lookup -> reuse best user-owned/core agent

If no match and allowed: create experimental reusable specialist with sanitized generic prompt

Assign tasks -> dependency graph -> parallel groups -> execute specialists

Specialists may call only allowlisted tools: READ_PROJECT_FILES and LOAD_PROJECT_MEMORY

Candidate changes are normalized, byte/file limited, accepted/rejected, then integrated by Python

Workflow success updates metrics and can promote agents; failures/safety violations can disable agents

Dynamic agent concept	Backend behavior
AgentDefinition	Stores id, name, role, capabilities, prompt, allowed tools, domains, lifecycle, owner, limits, metrics, schemas.
CapabilityTask	Represents a bounded unit of specialist work with capability, schemas, dependencies, risk, and runtime action.
AgentAssignment	Maps task to agent with assignment type, confidence, and reason.
WorkflowPlan	Contains tasks, assignments, dependency graph, parallel groups, active agents, created/reused ids, and completion proof.
Registry scoring	Scores by lifecycle, supported domain, required capability, success rate, and usage count.
Creation guard	Only RUN_DYNAMIC_SPECIALISTS tasks can create dynamic agents; core/Python-guarded capabilities cannot.
Prompt sanitization	Project-specific system prompts are replaced with generic reusable prompts.
Tool boundary	Forbidden tools and unknown tools are rejected. Direct file writes are disabled.
Execution	Specialists run in parallel groups with timeout, tool-call budget, and guarded tool executor.
Candidate limits	Candidate changes are limited by max files and max bytes per file before integration.
Lifecycle	States include core, experimental, reusable, and disabled. Successful runs promote; failures and safety violations lower trust.

Dynamic Agent Safety

- Dynamic agents cannot call WRITE_PROJECT_FILES, preview build, QA, sync, delete, or arbitrary tools.
- Model-provided project_id/user identity is ignored; Python binds identity from authenticated request context.
- Accepted candidate changes are integrated by backend code, then validated as a full project artifact.
- Safety violations and execution failures are logged to dynamic agent audit streams and lifecycle decisions.
- Persisted dynamic agents are user-scoped and reusable across that user's projects only.

Backend Tool Registry and Support

Tool	Backend support
READ_PROJECT_FILES	Reads supported website files from backend store; if linked local path exists, pulls local files and replaces store snapshot.
LOAD_PROJECT_MEMORY	Loads persisted memory items for project/user and optional namespace.
PERSIST_PROJECT_MEMORY	Upserts final project memory, summaries, workflow plans, and checkpoints.
WRITE_PROJECT_FILES	Replaces project files in store and pushes to linked local path if present.
VALIDATE_PROJECT_ARTIFACT	Checks generated_website contract, required fields, required src/App.jsx, allowed paths, section/theme/file safety.
BUILD_PROJECT_PREVIEW	Builds preview from current committed project files.
BUILD_STAGED_PROJECT_PREVIEW	Builds preview from candidate files before commit.
RUN_PREVIEW_VISUAL_QA	Checks staged preview status, build logs, browser render mode, and runtime failure markers.
SYNC_LOCAL_PROJECT	Pulls from or pushes to linked local workspace after path validation.

Backend Support Layers

Layer	Responsibility
Storage	Projects, files, versions, events, agent runs, messages, tool calls, memory, permissions, roles.
Local workspace	Allowed-root validation, path normalization, project import validation, disk read/write, local sync events.
Runtime preview	Creates staged or committed Vite preview builds and resolves preview assets.
Visual QA	Scans preview build output and optional browser render/runtime result.
Artifact validation	Normalizes and validates project artifact, paths, colors, React safety, and required files.
Code diff	Builds project file diff summaries and redacts code for audit output.
Audit logging	Writes JSONL query/tool and dynamic-agent events with redaction and correlation ids.
Failure normalization	Maps runtime exceptions into user-facing structured API errors.
Progress stream	Sends NDJSON progress events for generate-stream while generation runs.
Runtime persistence	Persists agent runtime output, messages, tool calls, memory events, local sync, and generated file metadata.

Generation vs Update Branches

Branch	Flow
Conversation	Route intent -> conversation response -> persist run/messages -> no file writes.
New generation	Read files -> load memory -> prompt analysis -> dynamic workflow -> planner -> specialists -> UX/accessibility -> code agent -> validate -> staged preview -> QA -> write -> memory.
Targeted/scoped update	Read files -> load memory -> update analysis -> optional dynamic planning -> scoped update agent -> validate changed artifact -> staged preview -> QA -> write -> memory.
Repair	On validation/preview/QA failure, repair agent or scoped retry runs within budget; previous project files are restored if the loop cannot complete safely.

Completion Proof

- Supervisor cannot finish until generated_website exists.
- Artifact validation must return valid.
- Staged preview must be ready.
- Visual QA must pass.
- Files must be committed through WRITE_PROJECT_FILES.
- Project memory must be persisted.

A2A, ADK, LangChain, and Telemetry

Projection/support	Role
A2A runtime	Builds handoff messages, acknowledgements, confidence scores, canonical fields, channel routing, and transcript validation.
Google ADK runtime	Projects the Python-owned runtime into ADK plan/tool/session/event metadata. It is not the source of truth for writes.
LangChain/LangGraph runtime	Projects runtime steps into graph-style nodes, thread config, messages, and validation metadata.
Telemetry	Correlates query events, tool calls, generation runs, agent runs, dynamic-agent lifecycle events, failures, and completion.
Audit redaction	Generated code and candidate patch bodies are not written to JSONL logs; previews are truncated/hashed and secrets are redacted.

Security and Authority Model

- Gemini can route, plan, propose tool calls, generate artifacts, and repair outputs.
- Python binds user/project identity and validates every backend tool call.
- Model-generated identity or path values are normalized or ignored where backend context is authoritative.
- Files are never committed until validation, staged preview, and QA pass.
- Dynamic agents are bounded to read/memory tools and cannot mutate disk or backend store directly.
- Rollback restores previous files when the final generation/update path cannot complete safely.

Operational Summary

The website builder is an enterprise-style AI generation runtime. The model provides reasoning, routing, planning, specialist output, code artifacts, and repair attempts. The backend provides the authority: authenticated context, storage, local workspace sync, tool validation, artifact validation, staged preview, visual QA, file writes, memory persistence, lifecycle management, and auditability.